

Sport G&C Tournaments — Sistema de Gestión de Torneos Deportivos

Sistema de consola en C++ para administrar torneos deportivos: equipos, jugadores, partidos, estadísticas y reportes. Toda la información se persiste en archivos binarios (`.bin`) usando acceso directo (registros de tamaño fijo).

1. Diagrama de estructuras y tamaño exacto (sizeof)

Tamaños medidos en una compilación de 64 bits (g++, `-std=c++17`). Pueden variar $\pm$ unos bytes según el compilador/plataforma por reglas de *padding/alignment*, pero el orden de magnitud y la lógica de cálculo son los mismos.

1.1 `ArchivoHeader` — 16 bytes

Encabezado que va al inicio de **todos** los archivos `.bin`.

Campo	Tipo
Bytes	
cantidadRegistros	int
4	
proximoID	int
4	
registrosActivos	int
4	
version	int
4	
TOTAL = 16	
bytes	

1.2 Jugador – 208 bytes

Campo	Tipo
Bytes	
ID	int
4	
idEquipo	int
4	
nombre	char[100]

100		
cedula	char[20]	
20		
posicion	char[25]	
25		
edad	int	
4		
numeroDorsal	int	
4		
fechaRegistro	char[11]	
11		
anotaciones	int	
4		
tarjetaAmarillas	int	
4		
tarjetasRojas	int	
4		
eliminado	bool	
1		
fechaCreacion	time_t	
8		
fechaUltimaModificacion	time_t	
8		
(padding de alineación)	-	
~7		
		TOTAL = 208
bytes		

1.3 Equipo — 624 bytes

Campo	Tipo
Bytes	
ID	int
4	
nombre	char[100]
100	
ciudad	char[150]
150	
entrenador	char[100]
100	
fechaRegistro	char[11]
11	
numJugadores	int
4	
puntos	int
4	
jugados	int
4	
victorias	int
4	
empates	int
4	
derrotas	int
4	
anotacionAFavor	int
4	
anotacionEnContra	int
4	
partidosIDs	int[50]

200	
cantidadPartidos	int
4	
eliminado	bool
1	
fechaCreacion	time_t
8	
fechaUltimaModificacion	time_t
8	
(padding de alineación)	-
~15	
	TOTAL = 624
bytes	

1.4 Anotacion / tarjetaAmarilla / tarjetaRoja – 20 bytes c/u

Las tres estructuras comparten el mismo diseño
(eventos de partido):

Campo	Tipo
Bytes	
idJugador	int
4	
minuto	int
4	

equipo	char[12]
12	
TOTAL = 20	
bytes	

1.5 Partido – 1496 bytes

Campo	Tipo
Bytes	
ID	int
4	
idEquipoLocal	int
4	
idEquipoVisitante	int
4	
fecha	
char[11]	11
estado	
char[12]	12
descripcion	
char[200]	200
anotacionesLocal	int
4	
anotacionesVisitante	int
4	
tarjetasAmaLocal	int

4		
tarjetasAmaVisitante	int	
4		
tarjetasRojasLocal	int	
4		
tarjetasRojasVisitante	int	
4		
anotaciones		
Anotacion[22]	440	
tarjetaA		
tarjetaAmarilla[30]	600	
tarjetaR		
tarjetaRoja[8]	160	
numAnotaciones	int	
4		
numtarjetaAma	int	
4		
numTarjetasRojas	int	
4		
eliminado	bool	
1		
fechaCreacion	time_t	
8		
fechaUltimaModificacion	time_t	
8		
(padding de alineación)	-	
~7		
		TOTAL =
1496 bytes		

1.6 Torneo — 216 bytes

Campo	Tipo
Bytes	
nombre	char[100]
100	
deporte	char[50]
50	
formato	char[20]
20	
fechaInicio	char[11]
11	
fechaFin	char[11]
11	
fechaCreacion	time_t
8	
fechaUltimaModificacion	time_t
8	
inicializado	bool
1	
(padding de alineación)	-
~7	

TOTAL = 216

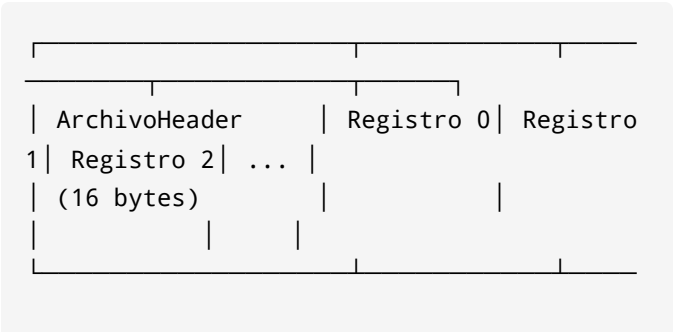
bytes

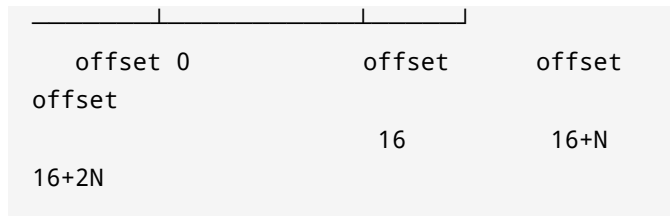
1.7 Resumen general

Estructura	sizeof (bytes)
ArchivoHeader	16
Jugador	208
Equipo	624
Anotacion	20
tarjetaAmarilla	20
tarjetaRoja	20
Partido	1496
Torneo	216

2. Fórmulas de cálculo de offset

Cada archivo binario (`equipos.bin` , `jugadores.bin` , `partidos.bin`) tiene la siguiente disposición:





Fórmula general usada en el código:

```
offset(indice) = sizeof(ArchivoHeader) +
indice * sizeof(Estructura)
```

Aplicada con `seekg / seekp` sobre el stream de archivo antes de leer o escribir el registro:

```
std::streampos posicion =
sizeof(ArchivoHeader) + indice *
sizeof(Equipo);
archivo.seekg(posicion, std::ios::beg);
archivo.read(reinterpret_cast<char*>
(&registro), sizeof(Equipo));
```

Ejemplos concretos (índice = 5)

Estructura	Cálculo	Offset resultante
Equipo	$16 + 5 \times 624$	3136 bytes
Jugador	$16 + 5 \times 208$	1056 bytes
Partido	$16 + 5 \times 1496$	7496 bytes

Este patrón permite **acceso directo (random access)** a cualquier registro sin recorrer secuencialmente el archivo completo: basta con conocer el índice del registro dentro del arreglo lógico.

Para operaciones de **búsqueda secuencial** o **listados completos**, en cambio, el programa se posiciona una sola vez después del header y va leyendo registro a registro:

```
archivo.seekg(sizeof(ArchivoHeader),
std::ios::beg);
while (archivo.read(reinterpret_cast<char*>
(&registroTemporal), sizeof(Jugador))) {
    // procesar registroTemporal
}
```

Para **inserciones**, el nuevo registro se agrega al final del archivo:

```
archivo.seekp(0, std::ios::end);
archivo.write(reinterpret_cast<const char*>
(&nuevoRegistro), sizeof(Equipo));
```

3. Manual de usuario básico

Al iniciar, el programa configura el idioma, crea la carpeta de datos (`bin/datos`) si no existe, e inicializa los archivos binarios con su respectivo header. Luego

solicita los datos iniciales del torneo (nombre, deporte, formato, fechas) si aún no existen.

Menú Principal

```

Sport G&C Tournaments

Torneo: <nombre del torneo>

Deporte: <deporte> | Formato:
<formato>

1. Gestión de Equipos
2. Gestión de Jugadores
3. Gestión de Partidos
4. Reportes
5. Mantenimiento
0. Salir

```

1. Gestión de Equipos

- Registrar nuevos equipos
- Buscar equipo (por ID / por nombre-subcadena)
- Actualizar equipo
- Listar equipos
- Eliminar equipo (baja lógica, campo `eliminado`)

2. Gestión de Jugadores

- Registrar jugador
- Buscar jugador
- Actualizar jugador
- Listar jugadores (todos o por equipo)
- Eliminar jugador (baja lógica)

3. Gestión de Partidos

- Programar partido
- Registrar resultado (goles, tarjetas amarillas/rojas por minuto)
- Buscar partido (por equipo / por ID)
- Listar partidos (todos o por estado: `PROGRAMADO` , `JUGADO` , `CANCELADO`)
- Cancelar partido

4. Reportes

- Tabla de posiciones
- Top 10 anotadores

- Top 10 tarjetas amarillas
- Top 10 tarjetas rojas
- Ficha técnica de un partido

5. Mantenimiento

- Verificar integridad referencial (detecta referencias rotas entre equipos, jugadores y partidos)
- Crear backups (copia los `.bin` a `../data/backups/`)

Notas de uso

- Todas las eliminaciones son **lógicas**: el registro permanece en el archivo, solo se marca `eliminado = true`.
- El sistema usa `int MI_ERROR = -1` como valor centinela de error en varias validaciones.
- Los archivos de datos se guardan en `../bin/datos/` y sus respaldos en `../data/backups/`.

4. Instrucciones de compilación y ejecución

Requisitos

- Compilador con soporte de **C++17** (g++ 8+ o equivalente, MSVC, clang).
- Sistema operativo: Windows, Linux o macOS (usa `<windows.h>` condicionalmente en Windows para funciones de consola).

Compilación (Linux / macOS)

```
g++ -std=c++17 -O2 -o sport_tournaments  
main.cpp
```

Compilación (Windows con MinGW)

```
g++ -std=c++17 -O2 -o sport_tournaments.exe  
main.cpp
```

Ejecución

Importante: el programa referencia rutas relativas como `../bin/datos/` y `../data/backups/`. Debe ejecutarse desde una carpeta llamada `bin` (o similar) ubicada un nivel dentro de la raíz del proyecto, de modo que `../bin/datos` y `../data/backups` sean válidas. Estructura típica esperada:

```
proyecto/  
├── bin/                <- ejecutar el  
binario desde aquí
```

```
|   └─ sport_tournaments
|   └─ datos/          <- se crea
automáticamente
└─ data/
    └─ backups/        <- debe existir o
crearse manualmente
```

```
cd proyecto/bin
./sport_tournaments
```

En Windows:

```
cd proyecto\bin
sport_tournaments.exe
```

Al primer arranque, el programa creará la carpeta `datos` y los archivos binarios vacíos (`torneo.bin` , `jugadores.bin` , `equipos.bin` , `partidos.bin`) con su `ArchivoHeader` inicial.